Parallel Ray Tracer

OpenMP + CUDA + Raylib

Complete Implementation Guide

Course: Introduction to Parallel Computing

Technologies: C++17 · CUDA · OpenMP · Raylib 5.5

Reference: Ray Tracing in One Weekend (v4.0.2, 2025) · NVIDIA CUDA Blog by Roger Allen

Project Overview

You are building a physically-based path tracer that renders a 3D scene of reflective and glass spheres on a ground plane. The same scene is rendered by three different backends, selectable at runtime by pressing a key. The output appears live in a Raylib window — pixels fill in progressively as threads complete their work.

The three backends are:

- Sequential — *one CPU thread, one pixel at a time. This is your correctness baseline.*
- OpenMP — *all CPU cores in parallel using #pragma omp parallel for. Directly exercises your OpenMP coursework.*
- CUDA — *thousands of GPU threads, one per pixel. The GPU renders the same scene orders of magnitude faster.*

The Raylib window sits on top: after each render, the completed pixel buffer is displayed instantly. The window also shows a live HUD — render time in milliseconds, speedup vs sequential, samples per pixel, and which mode is active. Your professor can see the benchmark without leaving the window.

What the final output looks like

At 1200×800 resolution with 100 samples per pixel on typical hardware:

Backend	Render time	Speedup
Sequential (1 thread)	8–15 minutes	1× baseline
OpenMP (8 threads)	60–90 seconds	~8–12×
CUDA (e.g. RTX 3060)	5–10 seconds	~50–100×

> *These numbers vary with your CPU and GPU. What matters is the relative comparison — and you measure and report your actual numbers in the writeup.*

Resources — bookmark these before starting

Resource	What it gives you	Where
Ray Tracing in One Weekend	The ray tracer itself, chapter by chapter	raytracing.github.io (free, browser)
Roger Allen's NVIDIA Blog Post	Chapter-by-chapter CUDA translation guide	developer.nvidia.com/blog/accelerated-ray-tracing-cuda

Resource	What it gives you	Where
rogerallen/raytracinginoneweekendincuda	Full CUDA reference code, one branch per chapter	github.com/rogerallen/raytracinginoneweekendincuda
define-private-public/PSRayTracing	Modern C++17 rewrite of the book — use for style reference	github.com/define-private-public/PSRayTracing
Raylib CMake Wiki	Official CMake integration guide	github.com/raysan5/raylib/wiki/Working-with-CMake
Modern CMake — OpenMP	Correct find_package(OpenMP) pattern	cliutils.gitlab.io/modern-cmake/chapters/packages/OpenMP.html
CUDA Toolkit Docs	nvcc flags, cudaMalloc, cudaMemcpy reference	docs.nvidia.com/cuda/cuda-c-programming-guide

Phase 0: Environment Setup — Day 1

Get all three toolchains building before writing a single line of ray tracing code.

Step 0.1 — Install prerequisites

On Ubuntu/Debian Linux (recommended — CUDA tooling is smoothest on Linux):

```
# C++ compiler and build tools
sudo apt update
sudo apt install -y build-essential cmake git

# Raylib system dependencies
sudo apt install -y libgl1-mesa-dev libx11-dev libxrandr-dev \
  libxcursor-dev libxi-dev libxinerama-dev

# Verify CUDA toolkit (should already be installed with your GPU driver)
nvcc --version
nvidia-smi
```

> *If nvcc is not found, download the CUDA Toolkit from developer.nvidia.com/cuda-downloads. Choose your OS, architecture, and the 'deb (local)' installer. Follow the post-install PATH instructions exactly — this is where most people get stuck.*

Step 0.2 — Project structure

Create this directory layout before writing any code. Having the structure right from day one prevents CMake headaches later.

```
parallel-raytracer/
├── CMakeLists.txt          # the single build file
├── src/
│   ├── main.cpp             # raylib window + mode switching + HUD
│   ├── scene.h              # vec3, Ray, Camera, Material, Sphere
│   ├── render_seq.h/.cpp    # sequential render loop
│   ├── render_omp.h/.cpp    # OpenMP render loop
│   └── render_cuda.h/.cu    # CUDA kernel
└── build/                   # cmake builds here (gitignore this)
```

Step 0.3 — CMakeLists.txt

This is the complete CMakeLists.txt. Copy it exactly, then adjust the CUDA architecture (sm_86 below is for RTX 30-series — check yours with nvidia-smi or the CUDA GPU docs).

```
cmake_minimum_required(VERSION 3.18)
project(ParallelRayTracer LANGUAGES CXX CUDA)
```

```
set(CMAKE_CXX_STANDARD 17)
set(CMAKE_CXX_STANDARD_REQUIRED ON)
set(CMAKE_CUDA_STANDARD 17)
set(CMAKE_CUDA_STANDARD_REQUIRED ON)

# ── Raylib via FetchContent (downloads and builds automatically) ──
include(FetchContent)
FetchContent_Declare(
  raylib
  GIT_REPOSITORY https://github.com/raysan5/raylib.git
  GIT_TAG        5.5
)
FetchContent_MakeAvailable(raylib)

# ── OpenMP ──
find_package(OpenMP REQUIRED)

# ── CUDA architecture (change sm_86 to match YOUR GPU) ──
# RTX 20-series: sm_75  |  RTX 30-series: sm_86  |  RTX 40-series: sm_89
set(CMAKE_CUDA_ARCHITECTURES 86)

# ── Source files ──
set(SOURCES
  src/main.cpp
  src/render_seq.cpp
  src/render_omp.cpp
  src/render_cuda.cu
)

add_executable(raytracer ${SOURCES})

# ── Link everything ──
target_link_libraries(raytracer PRIVATE
  raylib
  OpenMP::OpenMP_CXX
)

# ── Pass OpenMP flags through nvcc to host compiler ──
target_compile_options(raytracer PRIVATE
  $<$<COMPILE_LANGUAGE:CUDA>:-Xcompiler=${OpenMP_CXX_FLAGS}>
)

# ── CUDA: allow __host__ __device__ in headers included by .cpp files ──
set_target_properties(raytracer PROPERTIES
  CUDA_SEPARABLE_COMPILATION ON
)
```

Build commands:

```
cd parallel-raytracer
cmake -B build -DCMAKE_BUILD_TYPE=Release
cmake --build build -j$(nproc)
./build/raytracer
```

> *Run cmake -B build first with an empty src/ (just touch src/main.cpp etc.) to verify the build system works before you have any real code. Fix CMake errors now, not after writing 500 lines.*

Phase 1: Core Ray Tracer (Sequential) — Days 2–6

Follow the book chapter by chapter. Do not skip steps. Every chapter produces a visible result.

Follow Ray Tracing in One Weekend at raytracing.github.io. Everything in this phase goes into scene.h and render_seq.cpp. Do not add OpenMP or CUDA yet — correctness first.

Step 1.1 — vec3 and ray (Book Ch. 1–3)

The vec3 struct is the foundation of everything. Every position, direction, and colour in the ray tracer is a vec3. Write it in scene.h — but crucially, mark every method with __host__ __device__ so it compiles for both CPU and GPU later. This is the one change from the book's code that you need from day one.

```
// scene.h
#pragma once
#include <cmath>
#include <cuda_runtime.h>  // for __host__ __device__

struct vec3 {
    float x, y, z;

    __host__ __device__ vec3() : x(0), y(0), z(0) {}
    __host__ __device__ vec3(float x, float y, float z) : x(x), y(y), z(z) {}

    __host__ __device__ vec3 operator+(const vec3& b) const { return {x+b.x,
y+b.y, z+b.z}; }
    __host__ __device__ vec3 operator-(const vec3& b) const { return {x-b.x,
y-b.y, z-b.z}; }
    __host__ __device__ vec3 operator*(float t)        const { return {x*t,
y*t,   z*t  }; }
    __host__ __device__ vec3 operator*(const vec3& b)  const { return {x*b.x,
y*b.y, z*b.z}; }
    __host__ __device__ vec3 operator/(float t)        const { return *this *
(1.0f/t); }

    __host__ __device__ float dot(const vec3& b)  const { return x*b.x + y*b.y
+ z*b.z; }
    __host__ __device__ float length_sq()         const { return dot(*this); }
    __host__ __device__ float length()            const { return
sqrtf(length_sq()); }
    __host__ __device__ vec3  normalized()        const { return *this /
length(); }
    __host__ __device__ vec3  cross(const vec3& b) const {
        return { y*b.z - z*b.y, z*b.x - x*b.z, x*b.y - y*b.x };
    }
};
```

```
struct Ray {
    vec3 origin, dir;
    __host__ __device__ vec3 at(float t) const { return origin + dir * t; }
};
```

> *The __host__ __device__ markers are the key difference from the book. They tell nvcc 'compile this for both CPU and GPU.' Without them, your vec3 won't work inside a CUDA kernel.*

Step 1.2 — Camera and pixel output (Book Ch. 4, 7, 11)

Add a Camera struct to scene.h. Implement get_ray(x, y) which maps pixel coordinates to a ray from the camera's eye through that pixel. Also implement anti-aliasing here: get_ray adds a small random offset to x and y, so calling it multiple times per pixel gives you slightly different rays. Average the results.

For now, write pixels to a std::vector<Color> framebuffer (Color is raylib's RGBA struct). render_seq.cpp loops over all pixels and fills this buffer. Write it to a PPM file first to verify — PPM is plain text and needs no library.

```
// render_seq.cpp  — verify output with PPM before adding raylib
void render_sequential(Color* fb, int W, int H,
                       const Camera& cam, const Scene& scene,
                       int samples, int max_depth) {
    for (int j = 0; j < H; j++) {
        for (int i = 0; i < W; i++) {
            vec3 col = {0,0,0};
            for (int s = 0; s < samples; s++) {
                Ray r = cam.get_ray(i + random_float(), j + random_float(), W,
H);
                col = col + ray_color(r, scene, max_depth);
            }
            col = col * (1.0f / samples);
            fb[j*W + i] = { (u8)(255*col.x), (u8)(255*col.y), (u8)(255*col.z),
255 };
        }
    }
}
```

Step 1.3 — Spheres and hit detection (Book Ch. 5–6)

Add a Sphere struct and a ray_sphere_hit() function to scene.h. The hit function solves a quadratic equation — the book walks through the maths clearly. Store all spheres in a std::vector<Sphere> on the CPU side (later you'll copy this to GPU memory).

Implement a sky_color() function that returns a blue-to-white gradient based on ray direction. This is your background when rays don't hit anything.

```
struct HitRecord {
    float t;
    vec3  p;         // hit point
    vec3  normal;    // surface normal
    int   mat_id;    // index into materials array
```

```
};

struct Sphere {
    vec3  center;
    float radius;
    int   mat_id;
};

__host__ __device__
bool hit_sphere(const Sphere& s, const Ray& r, float t_min, float t_max,
HitRecord& rec) {
    vec3  oc = r.origin - s.center;
    float a  = r.dir.dot(r.dir);
    float b  = oc.dot(r.dir);
    float c  = oc.dot(oc) - s.radius*s.radius;
    float disc = b*b - a*c;
    if (disc < 0) return false;
    float root = (-b - sqrtf(disc)) / a;
    if (root < t_min || root > t_max) {
        root = (-b + sqrtf(disc)) / a;
        if (root < t_min || root > t_max) return false;
    }
    rec.t = root;
    rec.p = r.at(root);
    rec.normal = (rec.p - s.center) / s.radius;
    rec.mat_id = s.mat_id;
    return true;
}
```

Step 1.4 — Materials (Book Ch. 8–10)

Three material types cover the book's final scene: Lambertian (matte/diffuse), Metal (reflective with optional fuzz), and Dielectric (glass with refraction). The book derives the scatter() function for each — follow it carefully.

Store materials as a struct with a type enum and the relevant parameters (albedo, fuzz, refraction index). This flat struct layout is essential for CUDA — a vtable-based class hierarchy will not work on the GPU.

```
enum class MatType { LAMBERTIAN, METAL, DIELECTRIC };

struct Material {
    MatType type;
    vec3    albedo;   // colour for Lambertian and Metal
    float   fuzz;     // Metal only: 0 = mirror, 1 = very fuzzy
    float   ref_idx;  // Dielectric only: 1.5 for glass
};
```

> *The book uses virtual functions and shared_ptr<Material> for polymorphism. Do NOT do this. Virtual functions require a vtable, which does not work in CUDA device code. Use the flat struct + switch statement shown above instead. This is the main 'modernisation' your implementation makes.*

Step 1.5 — The recursive ray_color function (Book Ch. 7–10)

ray_color() is the heart of path tracing. It casts a ray, finds the closest hit, scatters a new ray based on the material, and recurses. The recursion depth is capped (typically 50 bounces) to prevent infinite loops for glass-in-glass situations.

Mark this function __host__ __device__ — it will run identically on CPU and GPU. This is where the magic happens: the same function works in your sequential loop, your OpenMP loop, and your CUDA kernel without any changes.

```
__host__ __device__
vec3 ray_color(const Ray& r, const Sphere* spheres, int n_spheres,
               const Material* mats, int depth, RNG& rng) {
    if (depth <= 0) return {0,0,0};

    HitRecord rec;
    float t_max = 1e30f;
    int   hit_idx = -1;

    for (int i = 0; i < n_spheres; i++) {
        HitRecord tmp;
        if (hit_sphere(spheres[i], r, 0.001f, t_max, tmp)) {
            t_max = tmp.t;
            rec   = tmp;
            hit_idx = i;
        }
    }

    if (hit_idx < 0) {
        // Sky gradient
        float t = 0.5f * (r.dir.normalized().y + 1.0f);
        return vec3{1,1,1}*(1-t) + vec3{0.5f,0.7f,1.0f}*t;
    }

    Ray   scattered;
    vec3  attenuation;
    scatter(mats[rec.mat_id], r, rec, scattered, attenuation, rng);
    return attenuation * ray_color(scattered, spheres, n_spheres, mats,
depth-1, rng);
}
```

> *CUDA does not support true recursion by default. Convert ray_color to an iterative loop (accumulate attenuation in a running product, loop instead of recurse). Roger Allen's CUDA blog post shows exactly how to do this conversion — it is the single most important code change when porting to CUDA.*

Step 1.6 — Raylib window and sequential display

Now connect the working ray tracer to a Raylib window. The pattern is: fill a Color array on the CPU, upload it to a Texture2D, draw the texture each frame. This is the display loop that all three backends will share.

```
// main.cpp  — simplified skeleton
#include "raylib.h"
#include "scene.h"
#include "render_seq.h"
#include "render_omp.h"
#include "render_cuda.h"

int main() {
    const int W = 1200, H = 800;
    InitWindow(W, H, "Parallel Ray Tracer");
    SetTargetFPS(60);

    // CPU pixel buffer — filled by whichever backend is active
    std::vector<Color> fb(W * H, {20, 20, 20, 255});

    // Raylib texture — updated from fb after each render
    Image   img     = { fb.data(), W, H, 1, PIXELFORMAT_UNCOMPRESSED_R8G8B8A8
};
    Texture2D screen = LoadTextureFromImage(img);

    Scene   scene  = build_final_scene();
    Camera  cam    = make_camera(W, H);
    int     mode   = 0;  // 0=seq, 1=omp, 2=cuda
    double  last_ms = 0;

    while (!WindowShouldClose()) {
        // Key switches mode and triggers a re-render
        if (IsKeyPressed(KEY_ONE))   { mode = 0; }
        if (IsKeyPressed(KEY_TWO))   { mode = 1; }
        if (IsKeyPressed(KEY_THREE)) { mode = 2; }
        if (IsKeyPressed(KEY_R) || IsKeyPressed(KEY_ONE)
                                || IsKeyPressed(KEY_TWO)
                                || IsKeyPressed(KEY_THREE)) {
            double t0 = GetTime();
            if (mode == 0) render_sequential(fb.data(), W, H, cam, scene, 10,
50);
            if (mode == 1) render_omp(fb.data(), W, H, cam, scene, 10, 50);
            if (mode == 2) render_cuda(fb.data(), W, H, cam, scene, 10, 50);
            last_ms = (GetTime() - t0) * 1000.0;
            UpdateTexture(screen, fb.data());
        }

        BeginDrawing();
            DrawTexture(screen, 0, 0, WHITE);
            // HUD overlay
```

```
            const char* modes[] = {"Sequential", "OpenMP", "CUDA"};
            DrawText(TextFormat("Mode: %s  [1/2/3 to switch]", modes[mode]),
10, 10, 20, YELLOW);
            DrawText(TextFormat("Last render: %.1f ms", last_ms), 10, 36, 20,
YELLOW);
            DrawText(TextFormat("FPS: %d", GetFPS()), 10, 62, 20, YELLOW);
            DrawText("Press R to re-render", 10, H-30, 18, GRAY);
        EndDrawing();
    }
    UnloadTexture(screen);
    CloseWindow();
}
```

> *Milestone: build and run after this step. You should see a window with the Cornell-style sphere scene rendered sequentially. It will be slow — that is expected and is the point. This is your v1 baseline.*

Phase 2: OpenMP Parallelisation — Days 7–8

The simplest parallel change. One pragma, one RNG fix, one scheduling decision.

Step 2.1 — The core pragma

Open render_omp.cpp. The function is identical to render_seq.cpp except for three changes: the pragma on the outer loop, thread-local random number generation, and writing to the framebuffer by index rather than sequentially.

```
// render_omp.cpp
#include <omp.h>
#include "render_omp.h"

// Each thread gets its own RNG — avoids data races on shared state
thread_local std::mt19937 rng_engine(std::random_device{}());
thread_local std::uniform_real_distribution<float> dist(0.0f, 1.0f);
static float thread_random() { return dist(rng_engine); }

void render_omp(Color* fb, int W, int H,
                const Camera& cam, const Scene& scene,
                int samples, int max_depth) {

    #pragma omp parallel for schedule(dynamic, 4) collapse(2)
    for (int j = 0; j < H; j++) {
        for (int i = 0; i < W; i++) {
            vec3 col = {0,0,0};
            for (int s = 0; s < samples; s++) {
                Ray r = cam.get_ray(i + thread_random(), j + thread_random(),
W, H);
                col = col + ray_color(r, scene.spheres.data(), scene.n_spheres,
                                      scene.mats.data(), max_depth,
thread_random);
            }
            col = col * (1.0f / samples);
            fb[j*W + i] = { (u8)(255*col.x), (u8)(255*col.y), (u8)(255*col.z),
255 };
        }
    }
}
```

Step 2.2 — Understanding the three OpenMP decisions

Every choice in the pragma is deliberate and explainable to your professor:

- **parallel for:** distributes loop iterations across available threads. Each thread gets a set of (j, i) pixel pairs to compute.

- **schedule(dynamic, 4):** pixels near the centre of the image hit more spheres and take longer. Dynamic scheduling means idle threads pick up more work instead of waiting. The chunk size of 4 means each thread grabs 4 iterations at a time — small enough for good load balance, large enough to avoid scheduling overhead.
- **collapse(2):** treats the 2D loop as a single 1D pool of W×H iterations. Without this, OpenMP only parallelises the outer j loop, leaving the inner i loop sequential per thread.
- **thread_local RNG:** if all threads shared one std::mt19937, they would race to read and write its state simultaneously, producing garbage pixels and a data race. thread_local gives each thread its own independent RNG seeded with a hardware random device — results are different each run but always correct.

Step 2.3 — Timing and progress

Add a progress counter using an atomic integer. This is thread-safe without a mutex and shows your professor you understand shared state:

```
#include <atomic>
std::atomic<int> rows_done{0};


// Inside the parallel for, after processing each row j:
int done = ++rows_done;
if (omp_get_thread_num() == 0 && done % 20 == 0) {
    // Only thread 0 prints — avoids garbled output
    fprintf(stderr, "\rProgress: %d/%d rows", done, H);
}
```

> *Milestone: press 2 in your window. The same scene renders faster. If your machine has 8 cores, it should be roughly 6–8x faster than the sequential baseline. Differences from exact 8x are due to Amdahl's Law (scene setup, memory bandwidth limits) — mention this in your writeup.*

Phase 3: CUDA Implementation — Days 9–14

The most work but also the biggest payoff. Follow Roger Allen's blog post chapter by chapter alongside the book.

Primary resource for this entire phase: developer.nvidia.com/blog/accelerated-ray-tracing-cuda by Roger Allen. Read the blog post first, then implement. The companion repo at github.com/rogerallen/raytracinginoneweekendincuda has one git branch per book chapter — check out the corresponding branch when you are stuck.

Step 3.1 — Memory model: what lives where

CUDA has two separate memory spaces: host (CPU RAM) and device (GPU VRAM). You cannot dereference a device pointer from CPU code, and vice versa. Before writing any kernel, understand this layout:

Data	Lives on	Allocated with	Notes
Sphere array	Device	cudaMalloc	Copied from CPU once at startup
Material array	Device	cudaMalloc	Same
Camera	Passed by value	kernel parameter	Small enough to pass directly
RNG states	Device	cudaMalloc	One curandState per thread
Output pixel buffer (d_fb)	Device	cudaMalloc	Written by kernel
Output pixel buffer (h_fb)	Host (CPU)	std::vector / malloc	cudaMemcpy'd from d_fb then given to raylib

Step 3.2 — Setting up device memory (render_cuda.cu)

```
// render_cuda.h  — interface (included by main.cpp)
#pragma once
#include "raylib.h"
#include "scene.h"
void render_cuda(Color* h_fb, int W, int H,
                 const Camera& cam, const Scene& scene,
                 int samples, int max_depth);

// render_cuda.cu  — implementation
#include <cuda_runtime.h>
#include <curand_kernel.h>
#include "render_cuda.h"
```

```
// Helper: check CUDA errors and print a useful message
#define CUDA_CHECK(call) do {                                          \
    cudaError_t err = (call);                                          \
    if (err != cudaSuccess) {                                          \
        fprintf(stderr, "CUDA error %s:%d  %s\n",                     \
                __FILE__, __LINE__, cudaGetErrorString(err));          \
        exit(1);                                                       \
    }                                                                  \
} while(0)

void render_cuda(Color* h_fb, int W, int H,
                 const Camera& cam, const Scene& scene,
                 int samples, int max_depth) {

    // ── Upload scene to GPU ──────────────────────────────────
    Sphere*   d_spheres; Material* d_mats;
    CUDA_CHECK(cudaMalloc(&d_spheres, scene.n_spheres * sizeof(Sphere)));
    CUDA_CHECK(cudaMalloc(&d_mats,    scene.n_mats    * sizeof(Material)));
    CUDA_CHECK(cudaMemcpy(d_spheres, scene.spheres.data(),
               scene.n_spheres*sizeof(Sphere), cudaMemcpyHostToDevice));
    CUDA_CHECK(cudaMemcpy(d_mats, scene.mats.data(),
               scene.n_mats*sizeof(Material), cudaMemcpyHostToDevice));

    // ── Allocate output buffer on GPU ────────────────────────
    Color* d_fb;
    CUDA_CHECK(cudaMalloc(&d_fb, W * H * sizeof(Color)));

    // ── Allocate RNG states (one per pixel/thread) ───────────
    curandState* d_rng;
    CUDA_CHECK(cudaMalloc(&d_rng, W * H * sizeof(curandState)));

    // ── Launch kernels ───────────────────────────────────────
    dim3 threads(16, 16);                    // 256 threads per block
    dim3 blocks((W+15)/16, (H+15)/16);      // enough blocks to cover image

    init_rng_kernel<<<blocks, threads>>>(d_rng, W, H, 42ULL);
    CUDA_CHECK(cudaDeviceSynchronize());

    render_kernel<<<blocks, threads>>>(d_fb, W, H, cam,
                                       d_spheres, scene.n_spheres,
                                       d_mats, samples, max_depth, d_rng);
    CUDA_CHECK(cudaDeviceSynchronize());

    // ── Copy result back to CPU for raylib display ───────────
    CUDA_CHECK(cudaMemcpy(h_fb, d_fb, W*H*sizeof(Color),
cudaMemcpyDeviceToHost));
```

```
    // ── Free GPU memory ──────────────────────────────────────────
    cudaFree(d_spheres); cudaFree(d_mats);
    cudaFree(d_fb); cudaFree(d_rng);
}
```

Step 3.3 — The RNG init kernel

Each thread needs its own random number state. The init_rng_kernel runs once before the render kernel to set up one curandState per pixel:

```
__global__ void init_rng_kernel(curandState* rng, int W, int H, unsigned long
long seed) {
    int x = blockIdx.x * blockDim.x + threadIdx.x;
    int y = blockIdx.y * blockDim.y + threadIdx.y;
    if (x >= W || y >= H) return;
    int idx = y * W + x;
    // Each thread gets a unique sequence by using its pixel index as the
sequence number
    curand_init(seed, idx, 0, &rng[idx]);
}
```

Step 3.4 — The render kernel

This is the CUDA version of your render loop. Structurally identical to render_omp — the difference is that the parallelism comes from the GPU's thread scheduler, not from OpenMP:

```
__global__ void render_kernel(Color* fb, int W, int H, Camera cam,
                              const Sphere* spheres, int n_spheres,
                              const Material* mats, int samples, int max_depth,
                              curandState* rng) {
    int x = blockIdx.x * blockDim.x + threadIdx.x;
    int y = blockIdx.y * blockDim.y + threadIdx.y;
    if (x >= W || y >= H) return;   // guard: image may not divide evenly by 16

    int idx = y * W + x;
    curandState local_rng = rng[idx];  // copy to local (faster register
access)

    vec3 col = {0, 0, 0};
    for (int s = 0; s < samples; s++) {
        float u = (x + curand_uniform(&local_rng)) / W;
        float v = (y + curand_uniform(&local_rng)) / H;
        Ray r = cam.get_ray_uv(u, v);   // normalised [0,1] version of get_ray
        col = col + ray_color_iterative(r, spheres, n_spheres, mats, max_depth,
local_rng);
    }
    col = col * (1.0f / samples);
```

```
    // Gamma correction: raise to 1/2 power to correct for display gamma
    col.x = sqrtf(col.x); col.y = sqrtf(col.y); col.z = sqrtf(col.z);

    fb[idx] = { (unsigned char)(255.99f*col.x),
                (unsigned char)(255.99f*col.y),
                (unsigned char)(255.99f*col.z), 255 };

    rng[idx] = local_rng;  // write back updated RNG state
}
```

Step 3.5 — ray_color as an iterative loop

As mentioned in Phase 1, CUDA does not support deep recursion. Convert ray_color to an iterative version. The logic is identical — you accumulate an attenuation colour and update the current ray each bounce:

```
__device__
vec3 ray_color_iterative(Ray r, const Sphere* spheres, int n,
                         const Material* mats, int max_depth,
                         curandState& rng) {
    vec3 attenuation = {1, 1, 1};  // running product of surface colours

    for (int depth = 0; depth < max_depth; depth++) {
        HitRecord rec;
        int hit = find_closest_hit(r, spheres, n, 0.001f, 1e30f, rec);

        if (hit < 0) {
            // Sky — return accumulated attenuation * sky colour
            float t = 0.5f * (r.dir.normalized().y + 1.0f);
            vec3 sky = vec3{1,1,1}*(1-t) + vec3{0.5f,0.7f,1.0f}*t;
            return attenuation * sky;
        }

        Ray  scattered;
        vec3 surface_color;
        scatter(mats[rec.mat_id], r, rec, scattered, surface_color, rng);
        attenuation = attenuation * surface_color;  // multiply in surface
colour
        r = scattered;                              // continue tracing
scattered ray
    }
    return {0, 0, 0};  // exceeded max depth — return black
}
```

> *Milestone: press 3 in the window. The same image appears, rendered dramatically faster. If something looks wrong (bright spots, wrong colours), check that your random number generation is using curand_uniform correctly and that gamma correction is applied.*

Phase 4: Benchmarking & Writeup — Days 15–18

Turn your working renderer into a course project by measuring and explaining everything.

Step 4.1 — Timing methodology

Measure wall-clock time for each backend using GetTime() from raylib (or std::chrono if you prefer). For CUDA specifically, use CUDA events for accurate GPU timing — they measure time on the GPU side excluding memory transfers:

```
// Accurate CUDA timing with events
cudaEvent_t start_evt, stop_evt;
cudaEventCreate(&start_evt);
cudaEventCreate(&stop_evt);

cudaEventRecord(start_evt);
render_kernel<<<blocks, threads>>>(...);
cudaEventRecord(stop_evt);
cudaEventSynchronize(stop_evt);

float kernel_ms = 0;
cudaEventElapsedTime(&kernel_ms, start_evt, stop_evt);
// kernel_ms is GPU compute time only, excludes cudaMemcpy
```

Step 4.2 — Experiments to run and report

These are the minimum experiments that make a convincing parallel computing writeup. Run each three times and average the results.

Experiment	What you vary	What you measure	What it shows
Strong scaling	Thread count: 1, 2, 4, 8, 16	Render time at 1200x800, 100 spp	OpenMP speedup curve vs Amdahl's Law
Resolution scaling	Width: 400, 800, 1200, 1600, 2400	Time for seq, OMP, CUDA at each	Parallelism advantage grows with problem size
Sample scaling	Samples: 10, 25, 50, 100, 200	Time for all three backends	Linear scaling — good sanity check
Schedule comparison	static vs dynamic, chunk 1/4/16	Render time for OpenMP	Dynamic wins due to load imbalance
CPU vs GPU breakdown	CUDA kernel time vs memcpy time	Both measured with CUDA events	Shows memcpy overhead is negligible

Step 4.3 — On-screen HUD for demo

Add these metrics to the Raylib window so they are visible in any screenshot or recording. This replaces having to open a terminal during your demo:

```
// Extended HUD — add inside the BeginDrawing() block
DrawRectangle(0, 0, 360, 130, {0, 0, 0, 160});  // semi-transparent background

DrawText(TextFormat("Mode:    %s  (1/2/3)", modes[mode]),  10, 10, 18, YELLOW);
DrawText(TextFormat("Threads: %d", omp_get_max_threads()),  10, 32, 18, WHITE);
DrawText(TextFormat("Render:  %.1f ms", last_ms),           10, 54, 18, WHITE);
DrawText(TextFormat("Speedup: %.1fx vs seq", seq_ms/last_ms),10, 76, 18, LIME);
DrawText(TextFormat("Samples: %d/px   Depth: %d", spp, md), 10, 98, 18, WHITE);
DrawText(TextFormat("Res:     %dx%d", W, H),                 10,120, 18,
WHITE);
DrawText("R = re-render  S = save PNG", 10, H-28, 16, DARKGRAY);
```

Step 4.4 — Save to PNG

Add a save shortcut using raylib's built-in screenshot function. One line is all it takes:

```
if (IsKeyPressed(KEY_S)) {
    // Saves current window contents as screenshot
    TakeScreenshot("render_output.png");
}
```

Phase 5: Stretch Goals — Optional

Each is self-contained. Pick any one if you have time.

Progressive refinement (high impact, low effort)

Instead of waiting for the full render before displaying, update the raylib window after every N rows are complete. Renders at 1 sample/pixel first (fast, noisy), then accumulates more samples. The image visibly sharpens in real time.

```
// In the OpenMP version: update texture every 10 rows
#pragma omp parallel for schedule(dynamic, 1)
for (int j = 0; j < H; j++) {
    // ... render row j ...
    if (j % 10 == 0) {
        #pragma omp critical
        { UpdateTexture(screen, fb.data()); }  // thread-safe texture update
    }
}
```

Thread heatmap overlay

Store omp_get_thread_num() for each pixel in a separate int array. When the user presses H, colour pixels by thread ID instead of by surface colour. Each thread's work region appears as a distinct colour patch — the most visually compelling parallelism demo possible.

BVH acceleration (significant performance boost)

The current renderer tests every ray against every sphere: O(n_spheres) per ray. A Bounding Volume Hierarchy reduces this to O(log n). The book's second volume (Ray Tracing: The Next Week) covers BVH in detail. For the CUDA version, flatten the BVH into an array — pointer-based trees cannot be passed to a CUDA kernel.

Quick Reference

Common errors and fixes

Error	Cause	Fix
'__device__' not allowed on function	scene.h included from .cpp without CUDA	Add #include <cuda_runtime.h> at top of scene.h
cudaErrorLaunchTimeout (error 6)	Kernel ran too long, OS killed it	Reduce samples or resolution. On Linux, disable the display timeout.
Kernel produces black image	RNG not initialised before render kernel	Ensure init_rng_kernel runs and DeviceSynchronize before render_kernel
Bright white fireflies in image	Colour values > 1.0 not clamped	Clamp each channel: col.x = fminf(col.x, 1.0f) before writing to buffer
OpenMP: garbled pixels	rand() shared across threads — data race	Replace with thread_local mt19937 as shown in Phase 2
CMake: cannot find OpenMP	Missing -fopenmp linking	Use target_link_libraries(... OpenMP::OpenMP_CXX) not just compile flags
CUDA: sm_XX not recognised	Wrong GPU architecture in CMakeLists	Check your GPU: nvidia-smi, then set CMAKE_CUDA_ARCHITECTURES correctly
Refraction produces dark artifacts	Schlick approximation sign error	Follow Roger Allen's glass implementation exactly from his blog post

Key commands

Command	What it does
cmake -B build -DCMAKE_BUILD_TYPE=Release	Configure for release (optimisations on)
cmake --build build -j$(nproc)	Build using all CPU cores
nvcc --version	Verify CUDA compiler is installed
nvidia-smi	Check GPU model and compute capability
OMP_NUM_THREADS=4 ./build/raytracer	Run with specific thread count for scaling tests
nvprof ./build/raytracer (or ncu)	Profile CUDA kernel execution

Resume line

> *"Implemented a physically-based path tracer in C++17/CUDA with a live Raylib preview window; built sequential, OpenMP, and CUDA rendering backends switchable at runtime, achieving ~10x speedup with OpenMP and ~80x with CUDA over sequential baseline at 1200x800 with 100 samples per pixel."*

Good luck! The key is: get it working sequentially first, then add parallelism. Never the other way around.